# Python Essentials Part – II

Shantanu Shubham | Software Engineer 2 @Intuit

Airtribe

# Meet your Instructor

Shantanu is a Software Engineer 2 at Intuit, working on QuickBooks Online and its modernization, where he builds scalable and extensible backend systems and SDKs using Kotlin, Spring Boot, AWS, and Kubernetes.
He brings a strong grasp of system design, cloud-native architectures, and real-world engineering trade-offs, with hands-on experience in Python, Django, Angular, React, and React Native, and a deep curiosity for AWS, Machine Learning, Data Science, and Cybersecurity.

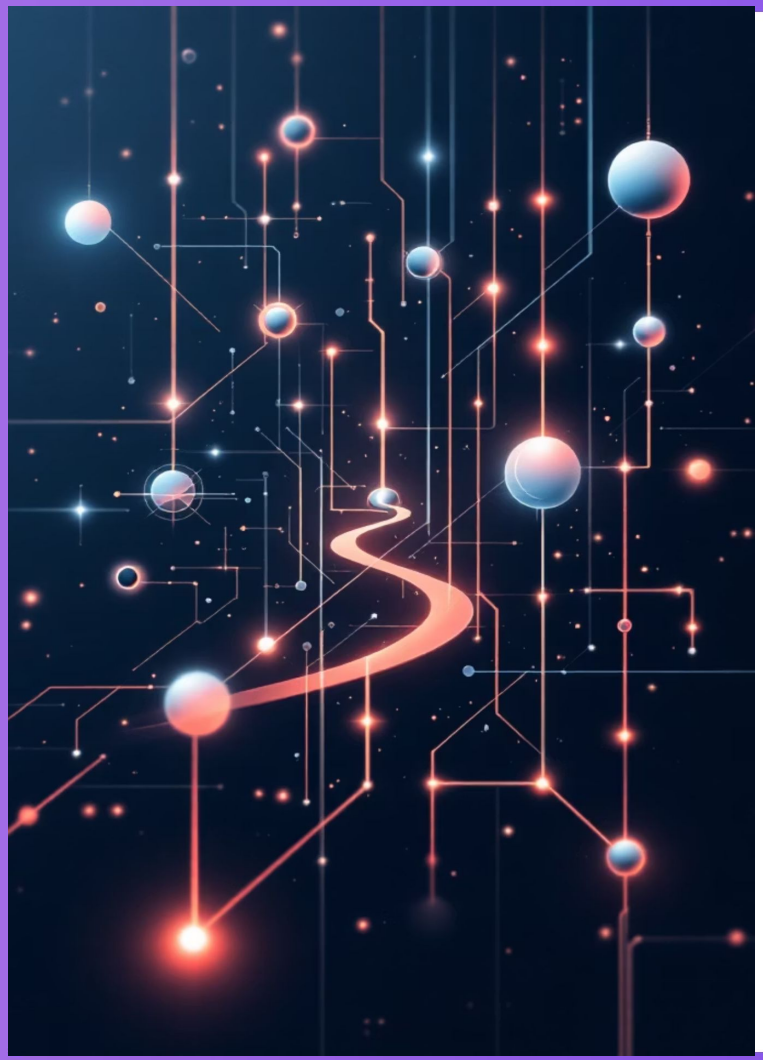
Shantanu Shubham

Software Engineer 2 @ INTUIT

Airtribe

# Agenda

- Why Python Matters for Backend
- Modules & Project Structure
- File Handling
- Exception Handling
- Virtual Environment + pip
- Type Hints
- Doubts



# Why This Session Matters

Before we move to OOP and backend systems…

You must understand:

- How real projects are structured
- How backend systems read and write data
- How to prevent code crashes
- How Python environments work
- Why modern teams use type hints

This session removes beginner confusion before we build serious systems next week.

Airtribe

# Session Roadmap

Today we will cover:

01

**Recap & syntax clarifications**

02

**Modules & project structure**

03

**File handling (JSON, CSV, logs)**

04

**Exception handling**

05

**Virtual environments & pip**

06

**Type hints**

By the end: You should be able to structure and run a mini backend-style project.

Airtribe

# Quick Recap

### What you already know:

- Variables
- Data types (list, dict, tuple, set)
- Functions
- Basic OOP
- Loops & conditionals

### Now ask:

- Any confusion in imports?
- Any confusion in function returns?
- Any confusion in data structures?

Clear now. After today, no syntax escalation.

# Why Backend Code Must Be Modular

In real companies:

- Codebase has 100+ files
- Multiple engineers work together
- Code must be reusable
- Code must be testable
- Code must be readable

Single-file scripts do not scale.

Modularity = Separation of concerns.

# What Is a Module?

A module = any Python file (.py)

**Example:**

| | | |
|---|---|---|
| math_utils.py | report_generator.py | database.py |

You import functions from one file into another.

# Creating a Module

**Example:**

**File: math_utils.py**

```
def add(a, b): return a + b
```

This is modular programming.

**File: main.py**

```
from math_utils import add
print(add(2, 3))
```

# Real Project Folder Structure

**Example backend project:**

```
project/
│
├── main.py
├── services/
│   ├── report.py
│   └── validator.py
├── utils/
│   └── helpers.py
└── data/
    └── transactions.json
```

### Why this structure?

- services → business logic
- utils → reusable helpers
- data → static files
- main.py → entry point

This is how production code looks.

Airtribe

# Exercise (Hands-on)

- Step 1: Write a function to calculate total transaction value.
- Step 2: Move it to a separate file.
- Step 3: Import and run from main.py.

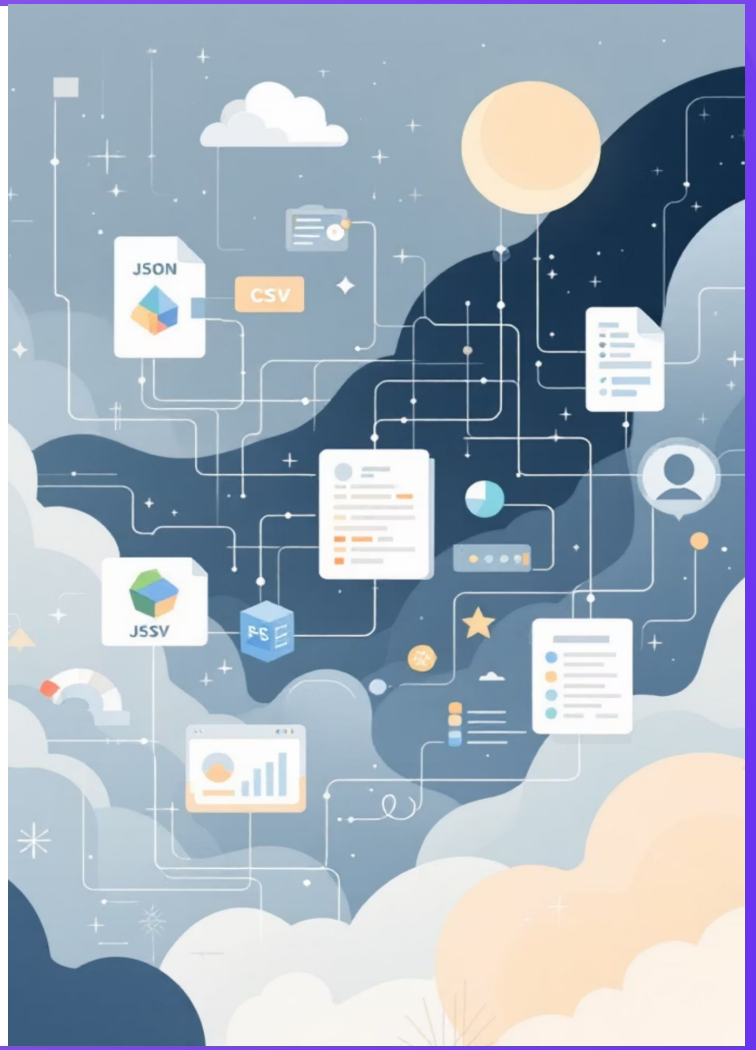
Goal: Feel how modular code behaves.

# Why File Handling Is Critical

Backend engineers constantly:

- Read user uploads
- Process CSV exports
- Store logs
- Read configuration files
- Generate reports

Everything is file-driven.



# Reading a File

```
with open("file.txt", "r") as file:
    content = file.read()
    print(content)
```

### Why use with?

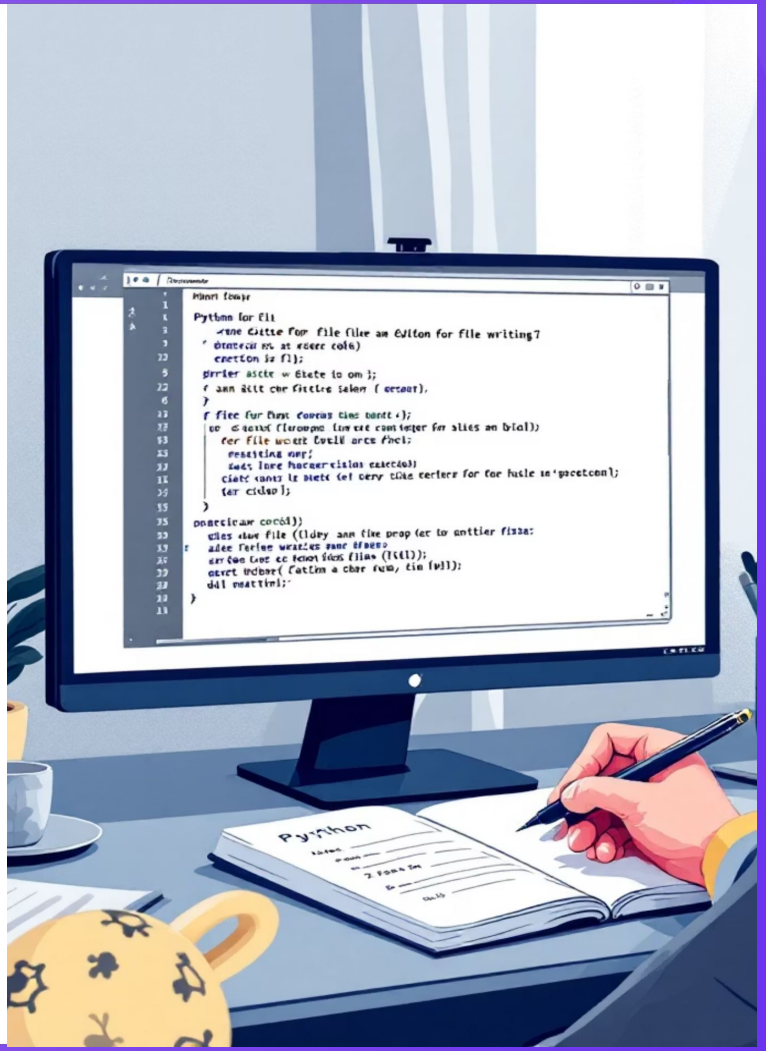
- Automatically closes file
- Prevents memory leaks
- Production best practice

# Writing to a File

```
with open("output.txt", "w") as file:
    file.write("Hello Backend World")
```

## Modes:

- "r" → read
- "w" → write (overwrite)
- "a" → append



# Working with JSON

Backend systems love JSON.

## Example:

```
import json
with open("data.json", "r") as file:
    data = json.load(file)
print(data)
```

## Why JSON?

- APIs communicate in JSON
- Structured format
- Easy to convert to dict

# Writing JSON

```
with open("output.json", "w") as file:
    json.dump(data, file, indent=4)
```

JSON = structured persistence.


# Working with CSV

CSV = spreadsheet data.

```
import csv
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

## Used for:

- Finance data
- Logs
- Data exports

Airtribe

# Live Build (Mini Backend Simulation)

## Problem:

You have a file: `transactions.json`

## Each entry:

- user
- amount
- type

## Task: Read file → Calculate total revenue → Write summary report to report.txt

This simulates:

Log processing in backend systems.

Airtribe

# Exception Handling

## Why Backend Code Must Never Crash

### In production:

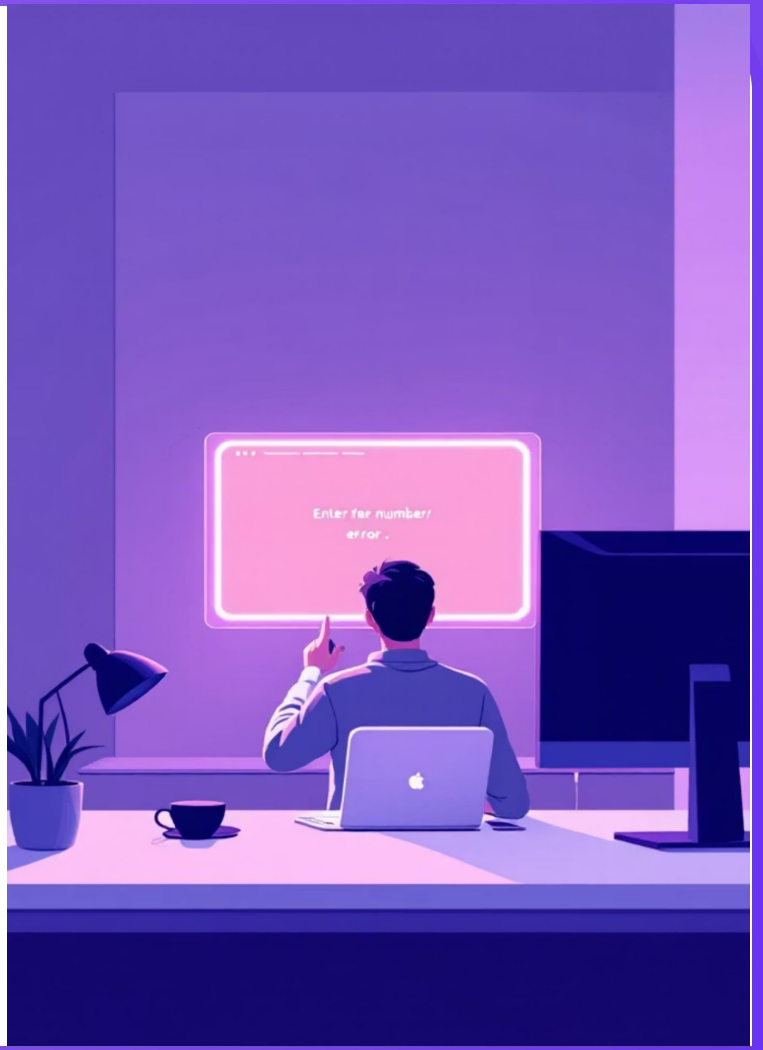
- Users send invalid data
- Files may not exist
- APIs may fail
- Data may be corrupted

If your backend crashes → system outage.

# Basic try/except

```
try:
    number = int(input("Enter number: "))
except ValueError:
    print("Invalid input")
```

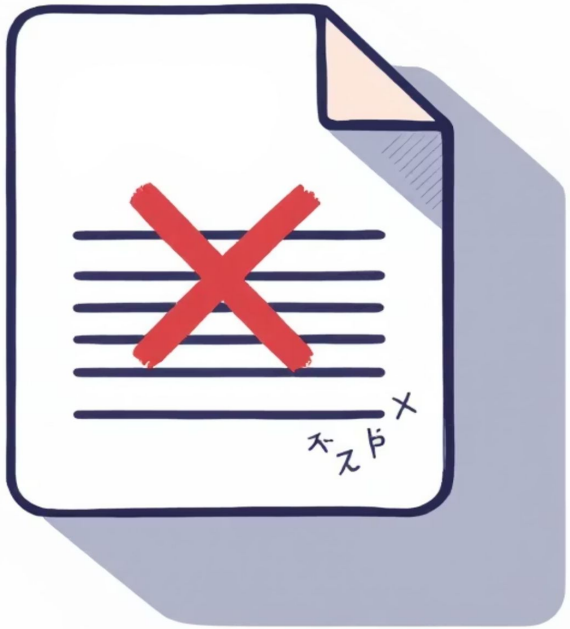
We control failure.



# Handling File Errors

```
try:
    with open("data.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found")
```

Backend code anticipates failure.



# Custom Exceptions

You can create your own errors.

```
class InvalidTransactionError(Exception):
    pass
```

## Raise it:

```
if amount < 0:
    raise InvalidTransactionError("Amount cannot be negative")
```

## Why?

- Cleaner error tracking
- Better debugging
- Clear business logic

Airtribe

# Virtual Environment + pip

## Why Virtual Environments Matter

**Problem:**

Project A uses Flask 2.0 Project B uses Flask 3.0

Without environments → chaos.

Virtual environment isolates dependencies.

# Creating Virtual Environment

Command line:

```
python -m venv venv
```

Activate:

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

You now have an isolated Python environment.

Airtribe

# Installing Packages

```
pip install requests
```

Check installed packages:

```
pip freeze
```

# requirements.txt

Save dependencies:

```
pip freeze > requirements.txt
```

Install from file:

```
pip install -r requirements.txt
```

This is how teams share environments.

# LIVE Activity

01

**Create new folder**

02

**Create venv**

03

**Install a package**

04

**Create requirements.txt**

05

**Delete environment**

06

**Recreate from requirements.txt**

Students must see dependency lifecycle.

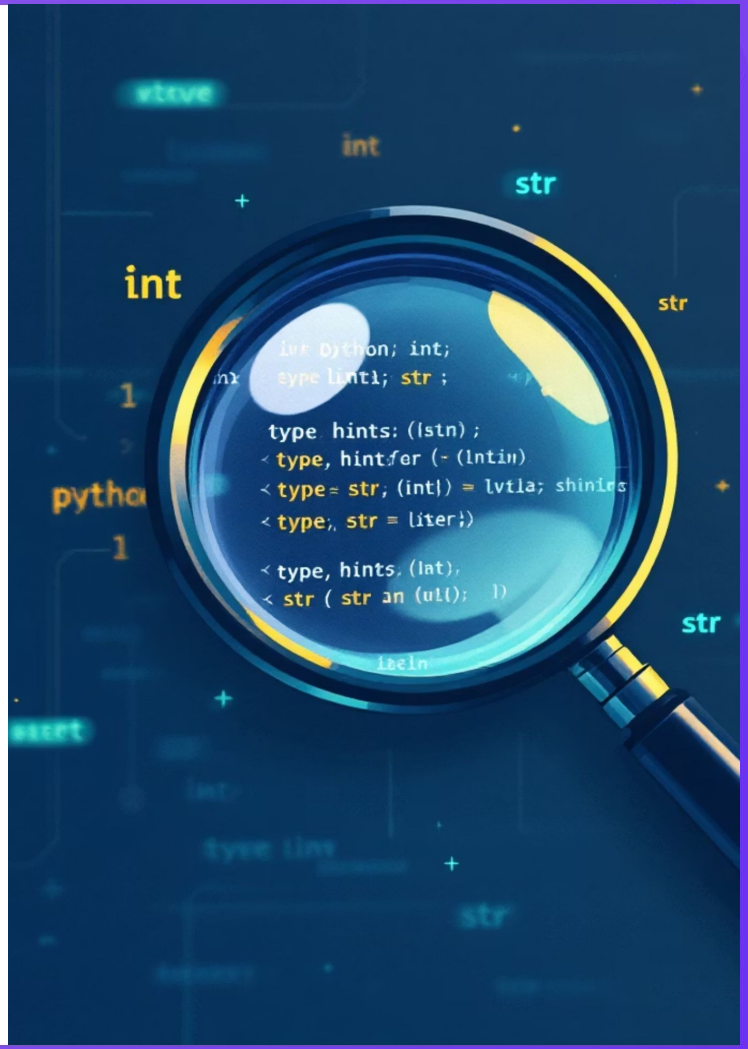
# Type Hints

## Why Modern Python Uses Types

As teams grow:

- Functions become complex
- Return types unclear
- Hard to debug

Type hints improve readability.



# Basic Function Typing

Without types:

```
def add(a, b): return a + b
```

With types:

```
def add(a: int, b: int) -> int: return a + b
```

Now anyone reading knows expectations.

# Why This Matters

Benefits:

- Better code clarity
- IDE suggestions
- Fewer runtime errors
- Prepares for larger systems

We will use types more next week.

# Final Wrap-Up

Today you learned:

- How real Python projects are structured
- How backend systems process files
- How to prevent crashes
- How environments isolate dependencies
- Why modern Python uses type hints

Next week: We build real OOP systems.

Now you are backend-ready.

# QnA

Airtribe

# Thank you

Airtribe